

JEGYZŐKÖNYV

Adatbázis rendszerek 2

féléves egyéni PL/SQL feladat

Készítette: Vágási Bence

Neptunkód: FNCCGA

Dátum:2026.05.05

Gyakorlatvezető: Dr. Bednarik László

Tartalom

1. A témakör ismertetése és megtervezése	3
2. A táblák sémaszerkezete.....	3
2.1 AUTOK tábla.....	3
2.2 JAVITASOK tábla	3
2.3 SZERVIZ_NAPLO tábla	3
3. A PL/SQL program kódja és megtervezése	3
3.1. Triggerek bemutatása	3
3.1.1 Kulcsgeneráló trigger.....	4
3.1.2 Módosítást kontrolláló trigger	4
3.1.3 Naplózó trigger (javitas_naplo_trg):.....	4
3.2 Csomagok, eljárások és kurzorok.....	5
4. Extra funkciók bemutatása.....	8
4.1 Dinamikus adatlekérdezés és változókezelés a tesztelés során	9
4.2 Összetett, üzleti logikát védő kivételkezelés	9
4.3 Részletes, többszintű naplózási mechanizmus:.....	9
5. Felhasznált irodalom és AI használat	9

1. A témakör ismertetése és megtervezése

A feladat során egy kliens-szerver architektúrájú, relációs adatbázisra épülő autószerviz-nyilvántartó alkalmazás szerveroldali logikáját valósítottam meg PL/SQL nyelven. Az adatbázis célja a szervizbe érkező autók alapadatainak (rendszer, márka, évjárat), valamint az elvégzett javítások adatainak (leírás, költség, dátum) tárolása és menedzselése. A rendszer tartalmaz egy naplózó modult is, amely minden adatmódosítási eseményt rögzít.

2. A táblák sémaszerkezete

Az adatbázis három fő táblából épül fel, melyek között egy-a-többhöz kapcsolat (1:N) valósul meg a fő- és kapcsolótábla között.

2.1 AUTOK tábla: A szervizelt gépjárművek adatait tárolja. Elsődleges kulcsa az `auto_id`, tartalmaz továbbá egyedi azonosítót (rendszer), szöveges, numerikus és dátum típusú mezőket is.

```
CREATE TABLE autok (  
    auto_id NUMBER PRIMARY KEY,  
    rendszam VARCHAR2(10) UNIQUE,  
    marka VARCHAR2(50),  
    gyartasi_ev NUMBER,  
    regisztracio_datum DATE  
);
```

2.2 JAVITASOK tábla: A gépjárműveken elvégzett munkákat rögzíti. A külső kulcs az `auto_id`, amely az autók táblára hivatkozik.

```
CREATE TABLE javitasok (  
    javitas_id NUMBER PRIMARY KEY,  
    auto_id NUMBER REFERENCES autok(auto_id),  
    leiras VARCHAR2(200),  
    koltseg NUMBER,  
    javitas_datum DATE  
);
```

2.3 SZERVIZ_NAPLO tábla: A triggerek által automatikusan kitöltött tábla, amely a biztonságot és a visszakövethetőséget szolgálja.

```
CREATE TABLE szerviz_naplo (  
    naplo_id NUMBER PRIMARY KEY,  
    muvelet VARCHAR2(50),  
    datum DATE,  
    felhasznalo VARCHAR2(50)  
);
```

3. A PL/SQL program kódja és megtervezése

A feladatkiírásnak megfelelően a szerveroldali logikát triggerekbe, valamint egy összefoglaló csomagba (Package) szerveztem, amelyek tartalmazzák a kurzorokat, a tárolt eljárásokat és a kivételkezelést is.

3.1. Triggerek bemutatása

A rendszerben három különböző trigger dolgozik:

3.1.1 Kulcsgeneráló trigger (**auto_pk_trg**): Beszúráskor ellenőrzi, hogy van-e elsődleges kulcs megadva, és ha nincs, egy szekvencia segítségével automatikusan generál egyet

```
CREATE OR REPLACE TRIGGER auto_pk_trg
BEFORE INSERT ON autok
FOR EACH ROW
BEGIN
    IF :NEW.auto_id IS NULL THEN
        :NEW.auto_id := auto_seq.NEXTVAL;
    END IF;
END;
```

3.1.2 Módosítást kontrolláló trigger (**javitas_ctrl_trg**): Üzleti logikát valósít meg. Meggátolja, hogy egy javítás költsége negatív érték lehessen, ilyen esetben saját hibaüzenetet (RAISE_APPLICATION_ERROR) dob.

```
CREATE OR REPLACE TRIGGER javitas_ctrl_trg
BEFORE INSERT OR UPDATE ON javitasok
FOR EACH ROW
BEGIN
    IF :NEW.koltseg < 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Hiba: A javítás költsége
nem lehet negatív!');
    END IF;
END;
```

3.1.3 Naplózó trigger (**javitas_naplo_trg**): A javitasok táblán történt INSERT, UPDATE és DELETE eseményeket (többféle esemény) rögzíti a naplótáblában a pontos dátummal és a felhasználó nevével.

```
CREATE OR REPLACE TRIGGER javitas_naplo_trg
AFTER INSERT OR UPDATE OR DELETE ON javitasok
FOR EACH ROW
BEGIN
    IF INSERTING THEN
```

```

        INSERT INTO szerviz_naplo (naplo_id, muvelet, datum,
felhasznalo)

        VALUES (naplo_seq.NEXTVAL, 'ÚJ JAVÍTÁS', SYSDATE, USER);

    ELSIF UPDATING THEN

        INSERT INTO szerviz_naplo (naplo_id, muvelet, datum,
felhasznalo)

        VALUES (naplo_seq.NEXTVAL, 'JAVÍTÁS MÓDOSÍTVA', SYSDATE,
USER);

    ELSIF DELETING THEN

        INSERT INTO szerviz_naplo (naplo_id, muvelet, datum,
felhasznalo)

        VALUES (naplo_seq.NEXTVAL, 'JAVÍTÁS TÖRÖLVE', SYSDATE,
USER);

    END IF;

END;

```

```

--- 6. KONTROLL TRIGGER (Módosítás kontrollálása) ---
A trigger blokkolta a műveletet: ORA-20001: Hiba: A javítás költsége nem lehet negatív!
ORA-06512: at "SYSTEM.JAVITAS_CTRL_TRG", line 3
ORA-04088: error during execution of trigger 'SYSTEM.JAVITAS_CTRL_TRG'

```

3.2 Csomagok, eljárások és kurzorok

A `autok_pkg` nevű csomag fogja össze a tábla funkcióit.

- A csomagban helyet kaptak az alapadatokat kezelő eljárások (Felvitel, Módosítás, Törlés).
- **Kivételkezelés:** Az `auto_felvitel` eljárás tartalmaz EXCEPTION blokkot, amely a duplikált rendszám (DUP_VAL_ON_INDEX) beszúrási kísérletét kapja el, és ROLLBACK parancsot hajt végre.
- **Kurzorok:** Két függvény készült. A `get_auto_marka` egy implicit kurzort használ az egyetlen rekord visszaadására, míg a `get_osszes_javitas_koltseg` egy explicit kurzor segítségével járja be a rekordokat, és aggregált (szummázott) értéket ad vissza az összes bevételről.

-- CSOMAG FEJLÉC (Specifikáció)

```
CREATE OR REPLACE PACKAGE autok_pkg AS
```

```

    PROCEDURE    auto_felvitel(p_rendszam    VARCHAR2,    p_marka
VARCHAR2, p_ev NUMBER);

```

```

        PROCEDURE    auto_modositas(p_id    NUMBER,    p_uj_rendszam
VARCHAR2);
        PROCEDURE auto_torles(p_id NUMBER);
        FUNCTION get_auto_marka(p_id NUMBER) RETURN VARCHAR2;
        FUNCTION get_osszes_javitas_koltseg RETURN NUMBER;
END autok_pkg;

```

-- CSOMAG TÖRZS (Implementáció)

```
CREATE OR REPLACE PACKAGE BODY autok_pkg AS
```

-- Kivételkezeléssel ellátott felvitel

```

        PROCEDURE    auto_felvitel(p_rendszam    VARCHAR2,    p_marka
VARCHAR2, p_ev NUMBER) IS
        BEGIN
                INSERT INTO    autok    (rendszam,    marka,    gyartasi_ev,
regisztracio_datum)
                VALUES (p_rendszam, p_marka, p_ev, SYSDATE);
                COMMIT;
        EXCEPTION
                WHEN DUP_VAL_ON_INDEX THEN
                        DBMS_OUTPUT.PUT_LINE('KIVÉTEL ELKAPVA: Ez a rendszám
(' || p_rendszam || ') már létezik az adatbázisban!');
                        ROLLBACK;
        END auto_felvitel;

```

-- Módosítás

```

        PROCEDURE    auto_modositas(p_id    NUMBER,    p_uj_rendszam
VARCHAR2) IS
        BEGIN

```

```

        UPDATE autok SET rendszam = p_uj_rendszam WHERE auto_id
= p_id;
        COMMIT;
    END auto_modositas;

```

-- Törlés

```

PROCEDURE auto_torles(p_id NUMBER) IS

```

```

BEGIN

```

```

    DELETE FROM autok WHERE auto_id = p_id;

```

```

    COMMIT;

```

```

END auto_torles;

```

-- Függvény: Egy rekord lekérdezése (Implicit kurzor)

```

FUNCTION get_auto_marka(p_id NUMBER) RETURN VARCHAR2 IS

```

```

    v_marka autok.marka%TYPE;

```

```

BEGIN

```

```

    SELECT marka INTO v_marka FROM autok WHERE auto_id =
p_id;

```

```

    RETURN v_marka;

```

```

EXCEPTION

```

```

    WHEN NO_DATA_FOUND THEN

```

```

        RETURN 'Nincs adat';

```

```

END get_auto_marka;

```

-- Függvény: Aggregált érték lekérdezése (Explicit kurzorral)

```

FUNCTION get_osszes_javitas_koltseg RETURN NUMBER IS

```

```

    v_osszesen NUMBER := 0;

```

```

    v_aktualis NUMBER;

```

```

    CURSOR c_koltsegek IS SELECT koltseg FROM javitasok;

```

```

BEGIN

    OPEN c_koltsegek;

    LOOP

        FETCH c_koltsegek INTO v_aktualis;

        EXIT WHEN c_koltsegek%NOTFOUND;

        v_osszesen := v_osszesen + NVL(v_aktualis, 0);

    END LOOP;

    CLOSE c_koltsegek;

    RETURN v_osszesen;

END get_osszes_javitas_koltseg;

END autok_pkg;

```

```

--- 1. ÚJ AUTÓ FELVITELE (Csomag eljárás) ---
Sikeresen felvéve: TESZT-01 Skoda Octavia.

PL/SQL procedure successfully completed.

--- 2. KIVÉTELKEZELÉS (Dupla rendszám felvitele) ---
KIVÉTEL ELKAPVA: Ez a rendszám (TESZT-01) már létezik az adatbázisban!

PL/SQL procedure successfully completed.

--- 3. ADATOK MÓDOSÍTÁSA (Csomag eljárás) ---
A 4-es azonosítójú autó rendszáma módosítva: MOD-001

PL/SQL procedure successfully completed.

--- 4. ADATOK TÖRLÉSE (Csomag eljárás) ---
Az 5-ös ID-jű Lada Niva sikeresen törölve.

PL/SQL procedure successfully completed.

--- 5. EGY REKORD LEKÉRDEZÉSE (Függvény) ---
Az 1-es azonosítójú autó márkája: Toyota Corolla

```

4. Extra funkciók bemutatása

4.1 Dinamikus adatlekérdezés és változókezelés a tesztelés során: A teszt szkriptben a módosítandó és törlendő rekordok azonosítóját (ID) nem fixen a kódba égetve (hardcoded) adtam meg. Ehelyett PL/SQL változókat (DECLARE) és implicit kurzorokat (SELECT ... INTO) használtam. A program a rendszám alapján dinamikusan keresi meg az adott autóhoz tartozó, szekvencia által generált kulcsot. Ennek köszönhetően a tesztelés robusztus, és bármikor, bármilyen szekvencia-állapot mellett hiba nélkül lefut.

4.2 Összetett, üzleti logikát védő kivételkezelés: A rendszer nem csupán az általános hibákat (WHEN OTHERS) kapja el. Az autók felvitelénél dedikáltan kezeli a redundáns adatok rögzítésének kísérletét (a DUP_VAL_ON_INDEX kivétel elkapása a csomagban). Ezen felül a módosításokat kontrolláló trigger (javitas_ctrl_trg) egyedi alkalmazáshibát generál a RAISE_APPLICATION_ERROR metódussal, ha a felhasználó érvénytelen (negatív) költséget próbálna rögzíteni.

4.3 Részletes, többszintű naplózási mechanizmus: A javitas_naplo_trg trigger feltételes szerkezet (IF-THEN-ELSIF) segítségével tesz különbséget a DML műveletek között (INSERTING, UPDATING, DELETING). Ennek megfelelően dinamikusan bejegyzést készít a szerviz_naplo táblába, amely nemcsak a művelet tényét, hanem a pontos serveridőt (SYSDATE) és az adatbázis-felhasználó nevét (USER) is rögzíti a maximális visszakövethetőség érdekében.

5. Felhasznált irodalom és AI használat

- Felhasznált környezet: Oracle Database, Visual Studio Code.
- A feladatkiírásban kért nyilatkozat: A feladat megtervezése, a PL/SQL szintaktika hibakeresése, valamint a kód strukturálása során mesterséges intelligenciát vettem igénybe. Becslésem szerint az AI használata az elkészítés során 80-20% történt